

Handoff Développeurs — Facebook / TikTok Scraper

Version PDF condensée. Préférer le HTML pour diagrammes et mise en page complète.

1. Vue d'ensemble

Système de scraping multi-plateforme (Facebook + TikTok) orienté métadonnées de posts + analyse IA Gemini, exposé via une API Gateway Spring Boot, orchestré par RabbitMQ, persisté dans MongoDB, packagé entièrement en Docker Compose .

Objectif TikTok : scraper un profil ou un CSV, streamer vite les résultats à l'utilisateur, enrichir ensuite avec Gemini, et gérer le re-scrape sans historique inutile.

Couche Techno

API / orchestration Spring Boot (gateway)

Messaging RabbitMQ

Persistence MongoDB

Workers Python + Playwright / Chrome

IA Google Gemini

Déploiement Docker Compose

2. Architecture globale

Schéma composants



- sticky IP



XX

■ Webshare ■■■■■■ TikTok ■

[illegible]

□ □ □ □ □ □ □ □ □ □ □ □ □ □

■ Gemini ■ (video_report)

□ □ □ □ □ □ □ □ □ □ □ □ □ □

Gateway upsert ■■■ MongoDB (1 doc / platform+postId)

Service Fait Ne fait pas

gateway API, jobs, Rabbit, upsert Mongo, TTL, cache reports Ne scrape pas TikTok

worker-tiktok Browser scrape, proxies, Gemini, rapports batch N'est pas la source de vérité Mongo

worker-facebook Scrape Facebook via sa queue —

rabbitmq Transport async, découplage —

mongodb Jobs + résultats Pas d'historique de versions

3. Décisions d'architecture (pourquoi)

Gateway + workers (pas monolithe) ADR

Contexte

Le scrape Playwright est long, fragile, CPU/RAM lourd.

Décision

API Spring séparée des workers Python.

Pourquoi

API responsive ; scale workers indépendamment ; isolation des crashes browser.

Conséquences

Évolutions résultats = souvent gateway + worker.

Alternatives rejetées

Scrape synchrone dans l'API = timeouts HTTP et blocage.

RabbitMQ (pas HTTP synchrone worker) ADR

Contexte

Besoin de buffer, retry, multi-plateformes.

Décision

Exchange scrape.exchange + queues dédiées + result queue + DLQ.

Pourquoi

Découplage, résilience, plusieurs consumers possibles.

Conséquences

Observer les files (UI :15672) en debug.

Alternatives rejetées

Appels HTTP gateway→worker : couplage fort, pas de buffer.

Mongo 1 doc / (platform, postId) ADR

Contexte

Besoin de l'état actuel d'une vidéo, pas d'un journal.

Décision

Upsert unique ; scrapeld rattache au dernier job.

Pourquoi

Simplifie cache Gemini et GET results.

Conséquences

Pas de timeline metrics historique (sauf évolution future).

Alternatives rejetées

Historique complet = complexité et coût stockage.

Streaming métadonnées puis Gemini ADR

Contexte

Gemini est lent/coûteux ; l'utilisateur veut voir tôt.

Décision

Publish RESULT immédiat, puis ENRICHMENT async.

Pourquoi

UX streaming + économie IA via cache video_report .

Conséquences

Deux messages Rabbit par post enrichi.

Alternatives rejetées

Attendre Gemini avant tout affichage = UX mauvaise.

Sticky proxy = sticky Chrome profile ADR

Contexte

Cookie global sur IP aléatoires → soft-block TikTok (grille vide).

Décision

sdwopfmy-N → tiktok_sessions/sdwopfmy-N/ persistent context.

Pourquoi

Cookies/fingerprint nés sous la même IP sticky.

Conséquences

Volume Docker obligatoire pour survivre aux recreate.

Alternatives rejetées

Un seul profil Chrome partagé = détection / locks.

Pool 20k + sample + blacklist ADR

Contexte

Beaucoup d'IP meurent (tunnel, auth, soft-block).

Décision

Sample N IP/job, blacklist ciblée, rotation fail-fast.

Pourquoi

Résilience sans dépendre d'une seule IP.

Conséquences

Surveiller proxy_blacklist.json pour éviter pool brûlé.

Alternatives rejetées

IP unique fixe = SPOF.

Chrome headed + Xvfb ADR

Contexte

Headless pur plus souvent détecté.

Décision

TIKTOK_HEADLESS=false + DISPLAY=:99 sous Xvfb.

Pourquoi

Fingerprint plus proche d'un navigateur réel en Docker.

Conséquences

Image plus lourde ; besoin Xvfb dans le container.

Alternatives rejetées

Headless seul = plus de soft-blocks observés.

TTL metrics 12h + exception null ADR

Contexte

Re-scrape fréquent ne doit pas écraser metrics fraîches, ni bloquer un 1er scrape sans likes.

Décision

Si metrics présentes et <12h → garder ; si null → accepter incoming.

Pourquoi

Équilibre fraîcheur / coût scrape.

Conséquences

Bug historique corrigé : TTL bloquait likes même quand null.

Alternatives rejetées

Toujours écraser = charge inutile ; jamais écraser = metrics figées null.

4. Déploiement Docker

Commande : docker compose up --build

scraper_gateway :8080

scraper_worker_tiktok / scraper_worker_facebook

scraper_rabbitmq :5672 + UI :15672

scraper_mongodb :27017

Volume host Rôle

tiktok_sessions/ Profils Chrome sticky persistants

webshare_residential_proxies.txt Pool ~20k IPs

proxy_blacklist.json Cooldownoldown IP

video_reports/ Artefacts Gemini / PDF

Piège : sans volume tiktok_sessions , chaque recreate perd les profils chauffés.

5. Flux métier : profil unique vs CSV

5.1 Profil unique

Séquence

User → Gateway: POST /scrape

Gateway → Mongo: job QUEUED

Gateway → Rabbit: task tiktok

Gateway → User: scrape_id

Rabbit → Worker: consume

Worker → TikTok: profile via sticky proxy

loop chaque vidéo

Worker → Rabbit: RESULT (text + metrics)

Rabbit → Gateway → Mongo: upsert

User → Gateway: GET results (déjà visibles)

loop enrichissement

Worker → Gateway: cache video_report?

alt hit: reuse

else: Gemini(description)

Worker → Rabbit: ENRICHMENT

Gateway → Mongo: merge video_report

Worker → Rabbit: COMPLETED

Gateway → Mongo: job SUCCESS

5.2 CSV batch

Lanes parallèles

CSV URLs

■

■■ Lane 1 (IP A) ■■ page ■■ page ■■ page

■■ Lane 2 (IP B) ■■ page ■■ page

■■ Lane N (IP N) ■■ page ■■ ...

■

■■ IP morte → blacklist + remplacement proxy

■■ posts streamés comme profil unique

■

■■ fin → rapport Gemini HTML/PDF → COMPLETED

Profil unique CSV

Proxies Sample + rotation séquentielle 1 IP dédiée / lane

Parallelisme 1 scrape (+ enrich async) N lanes

Rapport API + reports/post + rapport agrégé batch

6. Messaging RabbitMQ

Exchange : scrape.exchange

Queues jobs : scraping_queue_tiktok , scraping_queue_facebook

Résultats : scrape_result_queue

DLQ : scraping_queue_dlq

Le gateway publie selon platform . Les workers publient RESULT / ENRICHMENT / COMPLETED / ERROR.

7. Streaming & Gemini

Dès qu'une vidéo est trouvée → publish métadonnées + metrics (si dispo)

Après scrape → pour chaque post : cache DB video_report ou appel Gemini sur la description texte

Publish enrichment update → le client voit le report apparaître

Gemini est surtout text-only ici → confidence.level = low/medium est normal sans audio/visuel.

8. MongoDB — modèle & cache

ScrapeResult : 1 document par (platform, postId) . Champs clés : author, textContent, hashtags, metrics, videoReport, publishedAt, scrapedAt, scrapeld.

Règles upsert

Incoming result

■

■■ pas de doc (platform,postId) → INSERT

■■ doc existe → UPDATE même _id

■

■■ metrics présentes ET scrapedAt < 12h → garder metrics

■■ sinon (ou metrics null) → mergeMetrics(incoming)

■■ videoReport incoming ? merge : garder existant

Avant Gemini, le worker appelle l'API interne gateway pour récupérer les reports déjà connus et sauter l'analyse IA.

9. Pool proxies, sticky, blacklist, 0 posts

Modèle mental sticky

1 sticky = 1 IP = 1 Chrome sdwopfmy-42 → IP Webshare sticky → tiktok_sessions/sdwopfmy-42/ Sélection d'IP

Algorithme

pool 20k

→ drop blacklist

→ prioriser STICKY_POOL_SIZE (compléter si trop peu)

→ drop .in_use

→ prefer warmed > cold

→ shuffle + take PROXY_MAX_PER_JOB

→ pour chaque IP:

soft-block/auth → blacklist courte + next IP (pas de retry inutile)

tunnel/timeout → retry flaky puis blacklist longue

→ toutes échouent → FAILED count=0

Blacklist

Situation Blacklist ? Durée défaut

no_posts_found Oui 2h (soft)

challenge/captcha Oui 2h (soft)

403 / HTTP failure Oui 24h

timeout / tunnel Oui 24h

INVALID_AUTH_CREDENTIALS Oui 6h

SingletonLock / profile in use Non rotation soft

scrape réussi Non —

pause normale entre pages Non —

Fichier : platform/tiktok scraper/proxy_blacklist.json

0 posts : toutes les IP du sample ont échoué → status=FAILED , message friendly, count=0 .

10. Pipeline scrape TikTok

launch_persistent_context sur profil sticky

Warmup homepage (+ auto-warm si session froide)

Open profil + wait ancres /video/

Extract DOM + SIGI/universal data + réseau (hors For You)

Scroll jusqu'à max_posts / empty scrolls

Enrich : ouvrir pages /video/ pour stats / statsV2 / DOM counts

Return posts → worker stream + Gemini

Fail-fast : si aucune ancre /video/ après wait → soft-block → rotate (pas 2×25s inutiles).

11. Rapports métier

Par vidéo : video_report Gemini (thèmes, résumé, sentiment, keywords, confidence...)

Batch CSV : rapport agrégé Mauritanie 24h HTML + PDF (+ fallback si timeout Gemini)

12. Système de logs

Module logging_setup.py → JSON structuré : timestamp, level, logger, message, platform, service, scrape_id, url, post_id.

Soft proxy fail → WARNING (sans stack Playwright)

Bug inattendu → ERROR + traceback

Corréler via scrape_id

13. Problèmes rencontrés & solutions

Symptôme Cause Solution

Chrome profile in use SingletonLock / concurrence clear locks + .in_use + launch dans try

Profil OK, 0 vidéos Soft-block grille squelette wait /video/, parse HTML, sticky, fail-fast
Likes null API alors logs OK TTL 12h bloquait metrics null si metrics missing → merge incoming
Enrichissement partiel Tunnel pages vidéo retries + statsV2
Soft-block massif cookie ≠ IP 1 sticky = 1 Chrome profile
Build/perte sessions sockets / pas de volume dockerignore + volume sessions
Logs bruyants stack Playwright soft errors WARNING
Pool brûlé trop vite retries longs soft-block fail-fast + soft blacklist 2h
Gemini redondant pas de cache API interne reports + reuse

14. Carte du code

Arborescence Facebook_Scraper/

- docker-compose.yml
- .env
- logging_setup.py
- docs/
- ■■■■ HANDOFF_DEVELOPERS.md
- ■■■■ generate_handoff_report.py ← ce générateur
- scraper/ # Gateway Spring Boot
 - ■■■■ ../messaging/listener/ScrapeResultListener.java
- platform/
- tiktok scraper/
- ■■■■ worker.py
- ■■■■ scraper.py
- ■■■■ sticky_sessions.py
- ■■■■ video_analysis.py
- ■■■■ webshare_residential_proxies.txt
- ■■■■ proxy_blacklist.json
- ■■■■ tiktok_sessions/
- facebook scraper/

15. Guide debug

docker compose ps — services healthy
docker compose logs -f worker-tiktok
Chercher : Selected proxy sample / no_posts_found / enrichment ok likes=
Vérifier proxy_blacklist.json
Vérifier volume tiktok_sessions
Metrics null → logs enrich + TTL gateway
RabbitMQ UI :15672

16. Pièges à ne pas casser

Ne pas réinjecter cookies globaux sur IP aléatoires en mode sticky

Ne pas lancer 2 Chrome sur le même tiktok_sessions/<id>/

Ne pas traiter no_posts_found comme succès vide sans rotation

Ne pas relancer Gemini si video_report existe

Ne pas oublier le volume Docker des sessions

Ne pas logger les passwords proxies

Ne pas retirer l'exception TTL « metrics null »

17. Évolutions prioritaires

Healthcheck proxy avant d'ouvrir Chrome

Dashboard succès / blacklist / latence

Mode metrics-only refresh sans Gemini

Historique optionnel des metrics

Téléchargement média seulement si confiance basse

Tests intégration mock TikTok + TTL gateway

Alerte quota Webshare (INVALID_AUTH en rafale)

18. 10 tâches premier jour

Lire ce rapport + docker-compose + .env

Monter la stack, scraper un profil via Postman

Suivre un scrape_id dans les logs

Voir l'upsert Mongo (platform, postId)

Relancer <12h et observer reuse video_report

Observer rotation sur soft-block

Lire sticky_sessions.py + launch_persistent_context

Lire ScrapeResultListener.upsertResult

Lancer un petit CSV batch

Proposer une PR mineure (log / test / doc)